Neurocomputing 659 (2026) 131777

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

DiM: Improving multivariate time series forecasting with DI embedding and multi-head graph learning mechanism

Yipeng Mo [a], Haoxin Wang [a], Zuhua Yao [a], Chengteng Yang [a], Bixiong Li [b,d], Yiping Jiang [e], Songhai Fan [c,d], Site Mo [a,d,*]

[a] College of Electrical Engineering, Sichuan University, Chengdu, 610065, Sichuan, China
[b] College of Architecture and Environment, Sichuan University, Chengdu, 610065, Sichuan, China
[c] State Grid Sichuan Electric Power Research Institute, Chengdu, 610041, Sichuan, China
[d] State Key Laboratory of Intelligent Construction and Healthy Operation and Maintenance of Deep Underground Engineering, Xuzhou, China
[e] State Grid Sichuan Electric Power Company, Chengdu, 610041, Sichuan, China

HIGHLIGHTS

- Introduces a novel difference-inverted (DI) embedding strategy that integrates differenced embeddings with inverted embeddings, effectively capturing dynamic temporal patterns while maintaining computational efficiency.
- Proposes a multi-head graph learning mechanism to model time-varying inter-channel dependencies through dynamically adaptive graph structures, addressing the limitations of static graph representations in conventional GNNs.
- Develops DiM, a GNN-based forecasting framework that seamlessly integrates DI embedding and multi-head graph learning within the Metaformer architecture, achieving state-of-the-art performance on eleven benchmark datasets.
- Demonstrates empirical superiority, significantly outperforming existing methods in 53 out of 55 mean absolute error evaluations, validating the efficacy of the proposed temporal and relational modeling advancements.

ARTICLE INFO

Communicated by F.G. Guimaraes

Keywords:
Multivariate time series forecasting
Temporal dynamics
Inter-channel relationships
Graph learning

ABSTRACT

Accurate multivariate time series (MTS) forecasting is crucial for applications in traffic planning, energy management, financial investment, and healthcare. The challenge of forecasting MTS lies in managing the complex temporal dynamics and inter-channel relationships. However, despite the progress achieved by previous studies, they still fall short of adeptly addressing these complexities, leaving substantial scope for further refinement. To fill this gap, this paper proposes DiM, which seamlessly integrates the difference-inverted (DI) embedding strategy and the multi-head graph learning mechanism within the Metaformer framework. Specifically, the DI embedding employs a straightforward differencing operation to compensate for the limitations of previous inverted embedding methods in capturing temporal dynamics, thereby enhancing the model's ability to discern temporal patterns without significantly increasing computational complexity. The multi-head graph learning mechanism dynamically adjusts multiple graph structures to better represent the evolving relationships between channels, surpassing the constraints of static graph structures typically used in GNNs. The effectiveness of the DiM has been validated across eleven public datasets, where it achieved the best results in 53 out of 55 mean absolute error metrics compared to existing state-of-the-art models, establishing a new benchmark in MTS forecasting. Code is available at https://github.com/Yipengmo/DiM.

* Corresponding author at: College of Electrical Engineering, Sichuan University, Chengdu, 610065, Sichuan, China.
Email address: mosite@126.com (S. Mo).

https://doi.org/10.1016/j.neucom.2025.131777
Received 12 June 2025; Received in revised form 30 July 2025; Accepted 8 October 2025
Available online 18 October 2025
0925-2312/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

1. Introduction

Multivariate time series (MTS) forecasting is essential in diverse areas such as traffic planning [28,29], energy management [22,23], financial investment [26], and disease transmission [6]. Accurately predicting future values based on past observations is critical for informed decision-making, strategic planning, and policy development in these fields. MTS consists of multiple univariate time series, each representing a unique channel with distinct temporal dynamics. Additionally, these channels are interlinked through complex relationships. Therefore, the effectiveness of MTS forecasting hinges on the ability to capture temporal dependencies and model the intricate interactions between these channels.

Historically, statistical models like ARIMA and exponential smoothing have been popular in forecasting due to their simplicity and efficiency, but they are limited to univariate time series [3,27]. The emergence of deep learning models, particularly recurrent neural networks (RNNs) [12,17] and convolutional neural networks (CNNs) [2,7], has driven a shift towards a more sophisticated paradigm for capturing temporal dependencies. Despite the notable progress they have made in time series forecasting, the Markov assumption in RNNs and the local receptive property in CNNs potentially constrain their ability to model long-term dependencies effectively [10].

To overcome these challenges, Transformer-based models have been introduced for time series forecasting, especially for long-term forecasting. These models embed the channels at a specific timestamp into tokens, as depicted in Fig. 1(a), and employ attention mechanisms to capture global temporal dependencies, achieving significant success [20,33,45,46]. However, recent studies have demonstrated that a simple linear model can outperform Transformer-based models in most time series forecasting tasks, casting doubt on their effectiveness in MTS forecasting [39].

To reintroduce Transformer into the field of time series forecasting, extensive studies have been conducted. These efforts primarily focus on two aspects: embedding techniques and modeling the inter-channel relationships. Among these developments, the most representative model is the iTransformer [21]. Departing from the vanilla embedding techniques used in Transformer-based models, iTransformer adopts an inverted embedding strategy, where individual channels are treated as tokens to delineate temporal dependencies, as illustrated in Fig. 1(b). Additionally, it utilizes attention mechanisms to effectively capture the dynamics between channels, setting a new benchmark for state-of-the-art performance. Building on these advancements, we propose that refining embedding techniques to better represent temporal relationships and enhancing the modeling of inter-channel interactions could further improve the performance of MTS forecasting. Therefore, this paper aims to enhance MTS forecasting from these two essential perspectives.

Embedding methods. In order to explore more effective embedding methods, we revisit previous methods as outlined in Fig. 1. The vanilla embedding method utilizes linear or convolutional layers to aggregate information across various channels, effectively preserving the temporal characteristics of the time series after the embedding process. However, this channel-mixing structure is found to be vulnerable to the distribution drift, to the extent that its effectiveness is often inferior to simpler methods like linear models [11]. In response, iTransformer introduces the inverted embedding technique that maps the time series of each channel to a high-dimensional representation. While this method has contributed to significant performance improvements, it tends to overlook the inherent temporal properties of the time series, resulting in an inability to effectively capture the dynamic changes and sequential information within the time series. To address this limitation, MTPNet [41] introduces a dimension-invariant embedding approach that leverages CNNs to process MTS data while explicitly preserving both temporal dynamics and inter-channel relationships. Similarly, various temporal embedding methods based on attention mechanisms [25], temporal convolutional networks (TCNs) [36], and RNNs [38] have been extensively investigated to enhance temporal modeling capabilities. Despite their success, these methods frequently suffer from excessive computational overhead due to their inherently complex architectures. Therefore, it is urgent to develop an efficient embedding approach that effectively integrates temporal information while maintaining the robustness of previous methods. To bridge this gap, this paper proposes the difference embedding method, as illustrated in Fig. 1(c). Drawing inspiration from TDT loss [35], we apply differencing to the time series data prior to embedding. This approach provides a straightforward representation of temporal dependencies without significantly increasing additional parameters, thereby effectively compensating for the shortcomings of the inverted embedding method which lacks adequate temporal modeling.

Inter-channel relationships modeling. Another major challenge in MTS forecasting is to decouple the intricate dependencies among channels. Recent studies have explored various mechanisms such as CNNs [17], attention [21,31,42], multi-layer perceptrons (MLPs) [5,19] and graph neural networks (GNNs) [4,34,43] to model inter-channel relationships. However, the application of GNNs has encountered limitations due to their constrained expressive power. Indeed, GNNs struggle to model inter-channel relationships because the interactions between channels are not static but change over time [4]. Therefore, solely depending on a fixed graph structure is inadequate for capturing the diverse interdependencies and varying correlations among the channels. To overcome these bottlenecks, this paper introduces a novel multi-head graph learning mechanism, inspired by the multi-head attention mechanism [30]. Diverging from traditional methods that rely on a single graph structure, multi-head graph learning employs multiple predefined graph structures to represent the various potential dependencies among channels in MTS. These structures are dynamically updated during the training phase to more accurately reflect the intricate relationships among the channels.

Based on the above motivations, this paper presents DiM, a GNN-based model designed to enhance temporal and inter-channel modeling in MTS. DiM incorporates two principal components: difference-inverted (DI) embedding and the multi-head graph learning mechanism. These components are seamlessly integrated into the Metaformer framework [37], which replaces traditional attention mechanisms with a versatile token mixer. The contributions of this study are threefold:

- We propose a DI embedding strategy that synergizes difference embedding with the existing inverted embedding method, boosting the model's ability to capture temporal dynamics.
- Addressing the limitations of current graph learning, we have developed a multi-head graph learning mechanism that more accurately discerns inter-channel relationships, offering a robust alternative to traditional attention-based methods.
- We introduce DiM, a GNN-based model for MTS forecasting that incorporates DI embedding and the multi-head graph learning mechanism. The effectiveness and superiority of DiM have been validated across eleven public datasets.

2. Related works

2.1. Embedding methods for MTS forecasting

In the field of natural language processing, embedding techniques convert words into numerical vectors, significantly enhancing the ability of models to capture semantic and syntactic relationships between words [13]. When attempting to introduce Transformers into MTS forecasting, this concept is extended by mapping all variables at each time step into a high-dimensional space, as demonstrated by models such as Informer [45], Autoformer [33], Pyraformer [20], and FEDformer [46]. However, unlike words, time series data at individual points do not carry rich semantic information. Furthermore, variables from different sensors have distinct physical meanings, and simply merging them

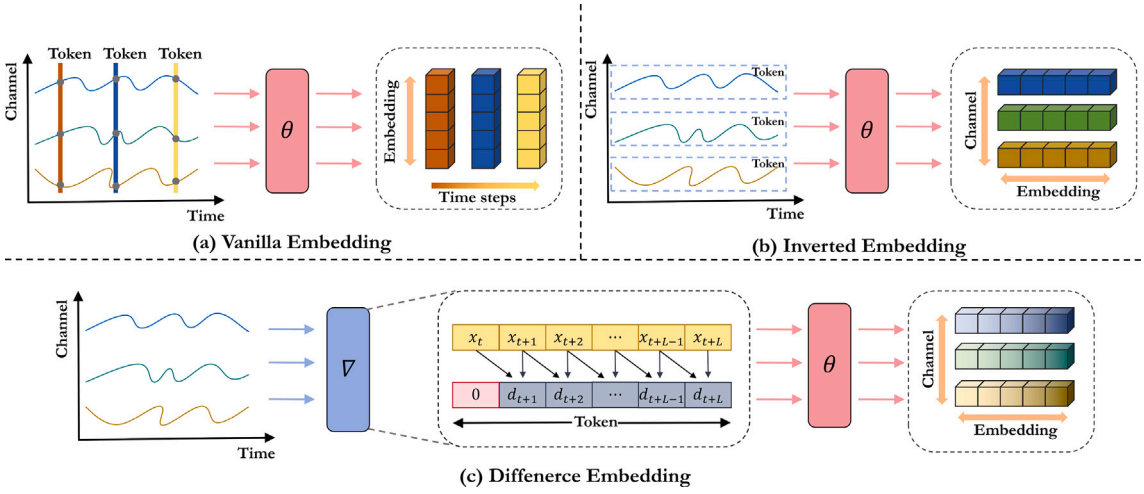


Fig. 1. Illustration of three different embedding strategies.

can introduce noise and reduce performance. To address these challenges, PatchTST [24] employs a patch embedding approach, which splits each series into several subsequences and treats each subsequence as a token. The iTransformer [21], on the other hand, proposes an inverted embedding method, treating the whole sequence as a token, which can be considered a special case of patch embedding. These methods, however, predominantly rely on linear mappings and thus may fail to capture temporal dynamics and sequential patterns. To enhance temporal modeling, researchers have introduced attention embedding [25], which leverages attention mechanisms to further extract temporal information. Complementary to this, convolutional approaches have emerged as alternatives: The sTransformer [36] builds on this by replacing linear mapping with a TCN, addressing the shortcomings of inverted embedding methods. Concurrently, MTPNet [41] proposes a dimension-invariant CNN embedding that explicitly preserves both temporal dynamics and spatial dependencies across channels through convolutional operations. Additionally, PRformer [38] leverages pyramidal RNN embedding to obtain time series representations that fuse multi-scale dependency information, enhancing the model's ability to capture complex temporal dynamics. Despite their benefits, these advanced techniques often result in increased computational demands.

2.2. Inter-channel relationships modeling for MTS forecasting

Decoupling the complex dependencies between channels is crucial for MTS forecasting, and several studies have focused on this area. LSTNet [17] employs CNN to capture inter-channel relationships and RNN to understand temporal dependencies. On the other hand, TSMixer [5] and MTS-Mixers [19] utilize MLPs to address both dependencies simultaneously. In contrast, Transformer-based models, such as iTransformer [21], Crossformer [42], and CSformer [31], leverage attention mechanisms to analyze channel relationships effectively. Recently, GNNs [16] have become popular for spatio-temporal forecasting tasks, such as traffic flow forecasting, due to their ability to handle structured data [40,44]. However, their application in MTS forecasting is limited by the requirement for a predefined graph structure, where each variable represents a node. This approach depends heavily on prior knowledge to define these structures, which can be impractical for MTS. Although some studies have explored learnable graph structures, a single graph structure often fails to capture the complex relationships between channels adequately, potentially diminishing the effectiveness of these models [4,34,43]. Therefore, this paper introduces a multi-head graph learning mechanism to address these limitations, enhancing the ability to capture complex inter-channel relationships in MTS and improving forecasting performance.

3. Methodology

3.1. Problem formulation

MTS forecasting focuses on predicting future values based on historical observations. Given a historical sequence $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_S\} \in \mathbb{R}^{S\times C}$ where S is the look-back window size and C is the number of channels, the objective is to predict the future values for the next T steps, denoted as $\mathbf{Y} = \{\mathbf{x}_{S+1}, \mathbf{x}_{S+2}, \ldots, \mathbf{x}_{S+T}\} \in \mathbb{R}^{T\times C}$. We aim to establish a mapping function $f(\cdot)$ from $\mathbf{X}$ to $\mathbf{Y}$, with the goal of minimizing the discrepancy between the predicted values and the ground truth. It is noteworthy that since the number of channels is constant across all time steps, for convenience, we denote $\mathbf{X}_{t,:} \in \mathbb{R}^C$ as the collection of time points recorded concurrently at step t, and $\mathbf{X}_{:,c} \in \mathbb{R}^S$ as the entire time series corresponding to the channel indexed by c.

In this paper, we introduce a multi-head graph learning mechanism to capture inter-channel relationships within the MTS. To clarify the graph-based approach, we provide key concepts here. From a graph perspective, each channel in the MTS is treated as a node in the graph structure, and the relationships between channels are characterized by an adjacency matrix. Specifically, a graph can be formalized as $\mathbf{G}=(\mathbf{V}, \mathbf{X}, \mathbf{A})$, where $\mathbf{V}$ is the set of nodes (the channels of the MTS), and $\mathbf{A} \in \mathbb{R}^{C\times C}$ represents the adjacency matrix of the graph. The element $\mathbf{A}_{ij}$ denotes the strength of the relationship between channel i and channel j. If $\mathbf{A}_{ij} = 0$, it indicates that channel i and channel j are independent. It is worth noting that the graph adjacency matrix is not typically provided by the MTS data and will be learned by our model.

3.2. Model architecture

The overall architecture of the DiM is illustrated in Fig. 2, which includes four main components: instance normalization, DI embedding, encoder, and projection.

In practical scenarios, datasets frequently demonstrate distribution drift due to external influences, where the probability distributions of the data evolve over time. This drift can adversely affect the generalization capability of models. To mitigate this issue, we initially apply instance normalization (IN) [14] to the input sequences, which can be formalized by:

$$\hat{\mathbf{X}} = \mathrm{IN}(\mathbf{X}) = \left\{ \frac{\mathbf{X}_{:,c} - \mu(\mathbf{X}_{:,c})}{\sigma(\mathbf{X}_{:,c})} \middle| c = 1, 2, \ldots, C \right\} \tag{1}$$

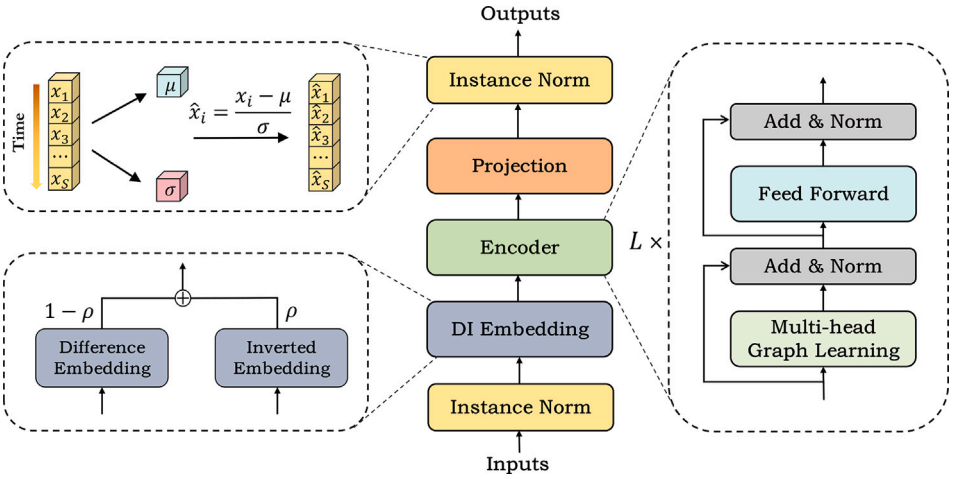


Fig. 2. The framework of the proposed DiM.

where $\mu(\cdot)$ and $\sigma(\cdot)$ denotes the mean and standard deviation, respectively, and $\hat{\mathbf{X}} \in \mathbb{R}^{S\times C}$ represents the outputs of IN. Following IN, DI embedding and encoder components are introduced to derive representations that capture both temporal dynamics and channel dependencies. The obtained representations then undergo a projection layer and reverse normalization to generate the predictions. This pipeline can be simply formulated as follows:

$$\begin{aligned}
&\mathbf{h}_c^{(0)} = \text{DI Embedding}\left(\hat{\mathbf{X}}_{:,c}\right),\\
&\mathbf{H}^{(l+1)} = \text{Encoder}\left(\mathbf{H}^{(l)}\right),\ l = 0, 1, \ldots, l-1,\\
&\hat{\mathbf{Y}}_{:,\mathbf{c}} = \text{Reverse IN}\left(\text{Projection}\left(\mathbf{h}_c^{(L)}\right)\right)
\end{aligned} \tag{2}$$

where $\mathbf{H} = \left\{\mathbf{h}_1, \ldots, \mathbf{h}_C\right\} \in \mathbb{R}^{C\times d}$ contains C embedded tokens of dimension d and the superscript denotes the encoder layer index, L is the number of encoder layers. Projection: $\mathbb{R}^d \mapsto \mathbb{R}^T$ is implemented by the MLP, and reverse IN refers to the inverse process of IN. $\hat{\mathbf{Y}} \in \mathbb{R}^{T\times C}$ represents the predicted values.

3.3. DI embedding

Difference embedding. Time series data inherently possess sequential dependencies, where the observation at the current time step is closely related to the preceding time step. Fully modeling these temporal dependencies is crucial for accurate time series forecasting. To reintroduce Transformers into the field of MTS forecasting, iTransformer employs inverted embedding that maps a sequence of length S into a d-dimensional space, serving as an alternative to the traditional vanilla embedding. However, this mapping strategy treats the input data simply as a vector and fails to capture the temporal dependencies, specifically the dynamic changes between consecutive time steps. Although many studies have explored RNNs, TCNs, and other embedding methods to enhance temporal modeling [25,36,38], these methods inevitably impose additional computational overhead. Inspired by the TDT loss [35], this paper introduces the difference embedding, as shown in Fig. 1(c). Specifically, for an input sequence $\hat{\mathbf{X}}_{:,c} \in \mathbb{R}^S$, the difference embedding can be expressed as:

$$\begin{aligned}
&\mathbf{D}_{:,c} = \text{Padding}(\nabla\hat{\mathbf{X}}_{:,c}),\\
&\mathbf{X}_{:,c}^{dif} = \text{Embedding}(\mathbf{D}_{:,c})
\end{aligned} \tag{3}$$

where the symbol '∇' represents the difference operator. $\mathbf{D}_{i,c} = \hat{\mathbf{X}}_{i,c} - \hat{\mathbf{X}}_{i-1,c} \in \mathbb{R}$ is the differencing result, $\text{Padding}(\cdot)$ is used to ensure the length of the differencing results matches the input time series, Embedding: $\mathbb{R}^S \mapsto \mathbb{R}^d$ can be realized through a linear mapping or convolutional layer, and $\mathbf{X}_{:,c}^{dif} \in \mathbb{R}^d$ refers to the results of difference embedding.

Advantages of the differencing operation. Unlike RNNs, TCNs, and other temporal modeling methods, the differencing operation does not introduce additional parameters or computational complexity. Furthermore, the differencing operation effectively performs a first-order derivative on the sequence, providing a straightforward representation of local dynamic changes. For instance, when the differencing result is positive, it indicates an increasing trend in the sequence, whereas a negative result suggests a decreasing trend. Critically, from a mathematical perspective, the differencing operation (∇) fundamentally transforms non-stationary time series. Consider a canonical non-stationary process, the random walk with drift, governed by $\hat{\mathbf{X}}_{\mathbf{t},\mathbf{c}} = \mu + \hat{\mathbf{X}}_{\mathbf{t-1},\mathbf{c}} + \epsilon_t$, where ϵ_t is white noise satisfying $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$. Applying the difference operator yields:

$$\mathbf{D}_{\mathbf{t},\mathbf{c}} = \nabla\hat{\mathbf{X}}_{\mathbf{t},\mathbf{c}} = \hat{\mathbf{X}}_{\mathbf{t},\mathbf{c}} - \hat{\mathbf{X}}_{\mathbf{t-1},\mathbf{c}} = \mu + \epsilon_t \tag{4}$$

This transformation systematically induces weak stationarity in the differenced series $\mathbf{D}_{:,\mathbf{c}}$, characterized by: (i) the expectation $\mathbb{E}[\mathbf{D}_{t,c}] = \mu$ remains constant for all t, (ii) the variance $\text{Var}(\mathbf{D}_{t,c}) = \sigma^2$ is time-invariant and finite, and (iii) the autocovariance function $\gamma_k = \text{Cov}(\mathbf{D}_{t,c}, \mathbf{D}_{t-k,c})$ depends only on the lag k [9]. By eliminating stochastic trends and transforming non-stationary processes into stationary time series, differencing facilitates the extraction of short-term temporal dependencies and high-frequency variations inherent in the innovations ϵ_t. In deep learning architectures, this induced stationarity promotes stable gradient propagation through embedding layers and mitigates representation distortion caused by evolving statistical properties over time [35]. This property is particularly crucial for effectively modeling the complex, non-stationary dynamics prevalent in MTS.

DI embedding. While the differencing sequence effectively captures local dynamic changes, it lacks the ability to encapsulate long-term temporal patterns. To provide a more comprehensive understanding of the underlying patterns in time series, we propose an integrated approach that combines the difference embedding method with the inverted embedding method. This integration is governed by a hyperparameter ρ, which finely tunes the balance between capturing immediate changes and recognizing enduring patterns. We refer to this method as the DI embedding, which can be expressed as:

$$\begin{aligned}
&\mathbf{X}_{:,c}^{inv} = \text{Embedding}(\hat{\mathbf{X}}_{:,c}),\\
&\text{DI Embedding}(\hat{\mathbf{X}}_{:,c}) = \rho\cdot\mathbf{X}_{:,c}^{inv} + (1-\rho)\cdot\mathbf{X}_{:,c}^{dif}
\end{aligned} \tag{5}$$

where $\mathbf{X}_{:,c}^{inv} \in \mathbb{R}^d$ represents the results of inverted embedding, and $\rho \in [0, 1]$ can be adjusted based on the characteristics of the specific dataset.

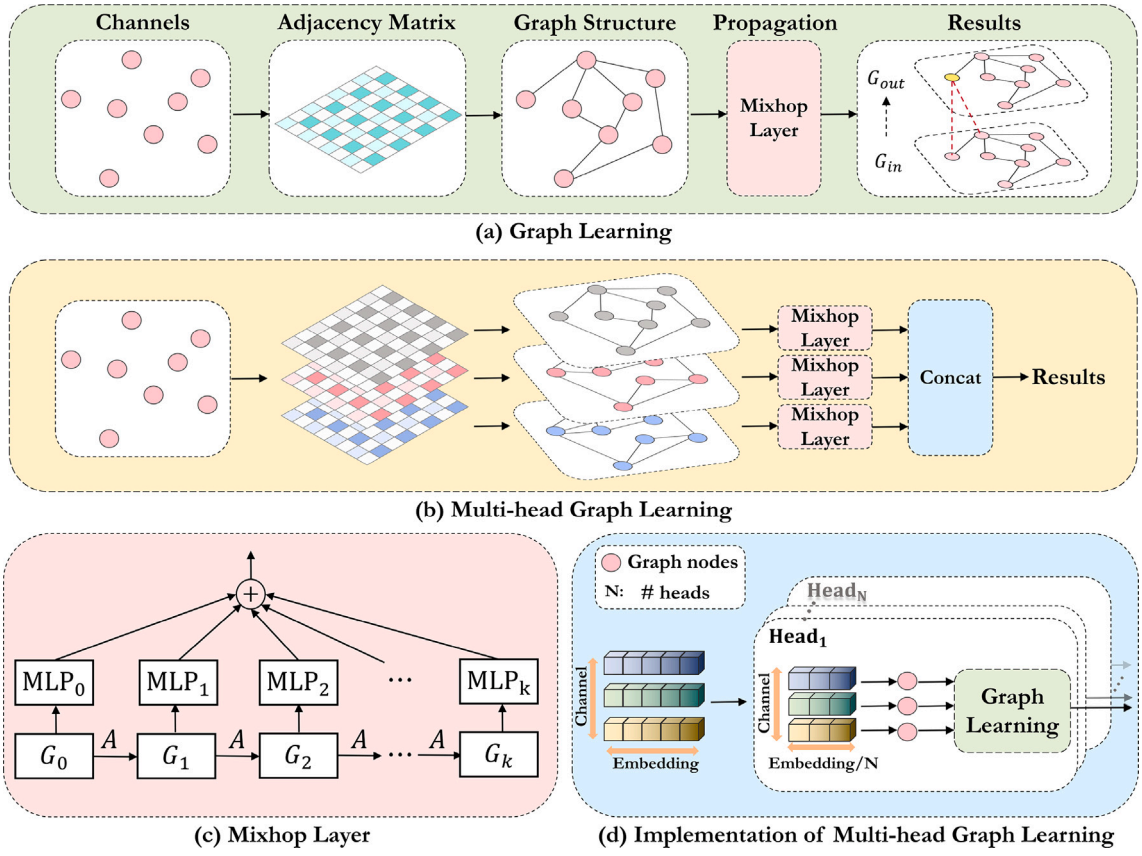


Fig. 3. Detailed overview of proposed multi-head graph learning mechanism.

3.4. Encoder

The embedding tokens are processed by DiM encoder layers to further capture the inter-channel dependencies. It is crucial to highlight that the encoder component of DiM adopts the Metaformer framework [37], which was first proposed in the field of computer vision. Metaformer suggests that the strength of Transformers derives not from its attention mechanism but from its overall structure. Therefore, the development of a more efficient token mixer could overcome the existing performance constraints of Transformers. In our approach, we employ a multi-head graph learning mechanism as the token mixer to enhance modeling capabilities.

3.4.1. Graph learning

To introduce the multi-head graph learning mechanism, we first present the concept of graph learning. The graph learning consists of graph structure learning and message passing, as illustrated in Fig. 3(a).

Graph structure learning. The key to graph structure learning is to determine the adjacency matrix to describe the dependencies between channels. In DiM, the adjacency matrix is learned in an end-to-end paradigm without the necessity of prior knowledge. Furthermore, in MTS forecasting, we expect the dependencies between channels to be unidirectional. For example, changes in temperature may affect the electricity consumption but the reverse does not hold. Therefore, following the approach of MTGNN [34], the learning process of the unidirectional graph structure can be represented by the following equation:

$$\begin{aligned} &\mathbf{M_1} = \tanh\left(\mathbf{E}\boldsymbol{\Theta}_1\right);\ \mathbf{M_2} = \tanh\left(\mathbf{E}\boldsymbol{\Theta}_2\right) \\ &\mathbf{A} = \mathrm{ReLU}\left(\mathbf{M_1}\mathbf{M_2^T} - \mathbf{M_2}\mathbf{M_1^T}\right). \end{aligned} \tag{6}$$

Here, $\mathbf{E} \in \mathbb{R}^{C\times D}$ denotes the randomly initialized node embeddings where D is the dimension of node embeddings, $\boldsymbol{\Theta}_1, \boldsymbol{\Theta}_2 \in \mathbb{R}^{D\times D}$ are learnable parameters. The subtraction method used in (6) facilitates the creation of a unidirectional graph structure, leading to the formation of the adjacency matrix $\mathbf{A} \in \mathbb{R}^{C\times C}$.

Message passing. After establishing a definitive graph structure, each node holds a latent vector. The purpose of message passing is to integrate information from interconnected nodes, thereby enhancing feature representation. From the perspective of MTS, message passing can selectively aggregate channel dependencies, merging channels with strong correlations and suppressing noise from irrelevant channels. For simplicity, in DiM, we employ the widely used MixHop method [1] for message passing, as shown in Fig. 3(c). Specifically, given an input $\mathbf{G}_{in} \in \mathbb{R}^{C\times d}$ and its corresponding adjacency matrix $\mathbf{A} \in \mathbb{R}^{C\times C}$, the process of MixHop can be represented by the following equation:

$$\mathbf{G}_{out} = \mathrm{Mixhop}\left(\mathbf{G}_{in}, \mathbf{A}\right) = \sum_{k=1}^{K} \tilde{\mathbf{A}}^k \mathbf{G}_{in} \mathbf{W}_k. \tag{7}$$

Here, K is the depth of graph propagation, $\tilde{\mathbf{A}} = \tilde{\mathbf{D}}^{-1}(\mathbf{A} + \mathbf{I})$ is the graph Laplacian matrix where $\tilde{\mathbf{D}} = 1 + \sum_j \mathbf{A}_{ij}$, and $\mathbf{W}_k \in \mathbb{R}^{d\times d}$ is a learnable weight matrix.

3.4.2. Multi-head graph learning mechanism

Graph learning is typically confined to understanding a singular graph structure. However, in MTS, the relationships between channels are intricate, where a singular graph structure is insufficient to fully represent channel dependencies. Inspired by the multi-head attention mechanism [30], we propose a multi-head graph learning mechanism that robustly models inter-channel relationships by learning graph structures across diverse latent spaces, as shown in Fig. 3(b). Formally, given the input $\mathbf{H}^{(l)}$ of the l-th layer, the multi-head graph learning mechanism can be represented as:

Table 1
Statistics of the datasets.

Datasets	Weather	Traffic	Electricity	ILI	ETTh	ETTm	Solar Energy	Flight	Tesla Stock
Number of Channels	21	862	321	7	7	7	137	7	6
Timesteps	52,696	17,544	26,304	966	17,420	69,680	51,579	26,304	1692
Temporal Granularity	10 mins	1 h	1 h	1 week	1 h	15 mins	10 mins	1 h	1 day

$$\mathbf{G}^{(l)} = \text{Concat}\left(\text{head}_1, \text{head}_2 \cdots, \text{head}_N\right)\mathbf{W}^o$$
$$where\ \ \text{head}_i = \text{Mixhop}\left(\mathbf{H}^{(l)}\mathbf{W}_i^H, \mathbf{A}_i\right) \tag{8}$$

In (8), N represents the number of heads. Besides, $\mathbf{W}^o \in \mathbb{R}^{d\times d}$ and $\mathbf{W}_i^H \in \mathbb{R}^{d\times d_N}$ are learnable parameter matrices where $d_N = d/N$. Since the dimension of each head is reduced, message passing within each graph can be efficiently performed in a smaller subspace, thereby reducing computational costs.

After fully modeling the channel dependencies through the multi-head graph learning mechanism, DiM adopts the conventional Metaformer protocol to further integrate information and enhance the expressive power of the model. This process can be expressed as:

$$\hat{\mathbf{G}}^{(l)} = \text{LayerNorm}\left(\mathbf{G}^{(l)} + \mathbf{H}^{(l)}\right)$$
$$\mathbf{H}^{(l+1)} = \text{LayerNorm}\left(\hat{\mathbf{G}}^{(l)} + \text{FeedForward}\left(\hat{\mathbf{G}}^{(l)}\right)\right) \tag{9}$$

where $\text{LayerNorm}(\cdot)$ represents the commonly adopted layer normalization method [18] and $\text{FeedForward}(\cdot)$ refers to the multilayer feedforward network.

4. Experiments

4.1. Experiment settings

4.1.1. Datasets

To validate the effectiveness of the proposed DiM, we conducted extensive experiments on nine mainstream datasets. These datasets cover various domains, including Weather, Traffic, Electricity, Influenza-Like Illness (ILI), Solar Energy and four Electricity Transformer Temperature (ETT) datasets, which are widely adopted benchmarks in MTS forecasting due to their diverse temporal patterns and practical relevance. To further assess the model's robustness in more complex and dynamic scenarios, we additionally incorporate the Flight and Tesla Stock datasets. The Flight dataset exhibits both periodicity and abrupt changes, reflecting complex temporal dynamics influenced by factors such as the COVID-19 pandemic. The Tesla Stock dataset, on the other hand, represents high-noise financial time series, and serves as a representative case for evaluating forecasting performance under strong volatility and uncertainty. Table 1 provides a summary of the key statistics for each dataset and the details of these datasets are as follows:

Weather[1]: This dataset consists of climate data for the year 2020 from a region in Germany, including 21 weather indicators such as air temperature and humidity, with data points collected every 10 min.

Traffic[2]: It collects hourly road occupancy data measured by 862 sensors on highways in the San Francisco Bay Area from January 2015 to December 2016.

Electricity[3]: The Electricity Consumption Load (Electricity) dataset records the electricity consumption of 321 customers, recorded hourly from 2012 to 2014, totaling 20,304 time steps.

ILI[4]: This dataset documents the percentage and total number of influenza-like illness patients collected weekly by the Centers for Disease Control and Prevention from 2002 to 2020.

ETT[5]: The ETT dataset (containing four sub-datasets) collected data from two counties in China over a period of two years, recording seven variables such as load and oil temperature. The temporal granularity for ETTh1 and ETTh2 is 1 h, while for ETTm1 and ETTm2, it is 15 min.

Solar Energy[6]: The Solar Energy dataset consists of solar power production records from 137 PV plants in Alabama, sampled every 10 min throughout the year 2006.

Flight[7]: The Flight dataset contains flight records from the OpenSky Network spanning from January 1, 2019, to December 31, 2021, with a temporal granularity of 1 h. It includes six variables related to air traffic. Notably, the dataset captures the impact of the COVID-19 pandemic on global flight activity.

Tesla Stock[8]: This dataset documents the historical daily price and volume data for TSLA (Tesla), collected from June 29, 2010, to March 17, 2017. The dataset includes variables such as Date, Open, High, Low, Close, Volume, and Adjusted Close, with prices adjusted for dividends and stock splits.

4.1.2. Baselines

We select the state-of-the-art (SOTA) model, iTransformer [21], along with nine popular models in MTS forecasting as our baselines. These include Transformer-based models: PatchTST [24], Crossformer [42], FEDformer [46], and Autoformer [33]; MLP-based models: TSMixer [5] and DLinear [39]; a CNN-based model: TimesNet [32]; and GNN-based models: MTGNN [34] and MSGNet [4]. It is worth noting that among these models, iTransformer, Crossformer, TSMixer, and MTGNN respectively utilize attention mechanisms, MLP, and GNN to model channel dependencies, while the other models tend to overlook inter-channel relationships to some extent.

4.1.3. Experimental settings

For the baseline models TSMixer, MTGNN, and MSGNet, we adhere to the results presented in their original publications or utilize their official code for replication. Results for the other baseline models are obtained from Ref. [21]. To facilitate a fair comparison, we set the look-back window to 36 for the ILI and Tesla Stock datasets, and 96 for the other datasets. In line with previous studies, all models are configured with consistent prediction lengths, with $T \in \{24, 36, 48, 60\}$ for the ILI and Tesla Stock datasets, and $T \in \{96, 192, 336, 720\}$ for the remaining datasets. For model training and validation, we follow the settings in iTransformer. Specifically, the training, validation, and testing ratio for the ETT dataset is set at 6:2:2, whereas the other datasets follow a ratio of 7:1:2. We utilize Mean Squared Error (MSE) and Mean Absolute Error (MAE) as the evaluation metrics to assess model performance.

4.1.4. Implementation details

In our study, we utilize the MSE as the loss function and employ the Adam optimizer [15] with an initial learning rate selected from the set $\{5\times10^{-3}, 10^{-3}, 2\times10^{-4}\}$ to effectively minimize the MSE. All experiments are conducted using PyTorch and executed on a single NVIDIA GTX 3090 24GB GPU. We set the ratio parameter ρ, which governs the inverted embedding and difference embedding, to 0.5. The dimension

[1] https://www.bgc-jena.mpg.de/wetter/
[2] https://pems.dot.ca.gov/
[3] https://archive.ics.uci.edu/dataset/321/electricityloaddiagrams20112014
[4] https://gis.cdc.gov/grasp/fluview/fluportaldashboard.html
[5] https://github.com/zhouhaoyi/ETDataset
[6] https://www.nrel.gov/grid/solar-power-data
[7] https://opensky-network.org/
[8] https://www.nasdaq.com/market-activity/stocks/tsla

Table 2
Full results for the multivariate forecasting task. We compare extensive competitive models under different prediction lengths following the setting of iTransformer [21]. *Avg* means the average results from all four prediction lengths.The best results are in **bold** and the second best are underlined.

Models		**DiM (Ours)**		iTransformer 2024		PatchTST 2023		Crossformer 2023		FEDformer 2022		Autoformer 2021		MSGNet 2024		MTGNN 2020		TSMixer 2023		DLinear 2023		TimesNet 2023	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTm1	96	**0.314**	**0.353**	0.334	0.368	0.329	0.367	0.404	0.426	0.379	0.419	0.505	0.475	0.319	0.366	0.381	0.415	0.323	0.363	0.345	0.372	0.338	0.375
	192	**0.355**	**0.377**	0.377	0.391	0.367	0.385	0.450	0.451	0.426	0.441	0.553	0.496	0.376	0.397	0.442	0.451	0.376	0.392	0.380	0.389	0.374	0.387
	336	**0.384**	**0.400**	0.426	0.420	0.399	0.410	0.532	0.515	0.445	0.459	0.621	0.537	0.417	0.422	0.475	0.475	0.407	0.413	0.413	0.413	0.410	0.411
	720	**0.447**	**0.439**	0.491	0.459	0.454	**0.439**	0.666	0.589	0.543	0.490	0.671	0.561	0.481	0.458	0.531	0.507	0.485	0.459	0.474	0.453	0.478	0.450
	Avg	**0.375**	**0.392**	0.407	0.410	0.387	0.400	0.513	0.496	0.448	0.452	0.588	0.517	0.398	0.411	0.457	0.462	0.398	0.407	0.403	0.407	0.400	0.406
ETTm2	96	**0.172**	**0.253**	0.180	0.264	0.175	0.259	0.287	0.366	0.203	0.287	0.255	0.339	0.177	0.262	0.240	0.343	0.182	0.266	0.193	0.292	0.187	0.267
	192	**0.235**	**0.295**	0.250	0.309	0.241	0.302	0.414	0.492	0.269	0.328	0.281	0.340	0.247	0.307	0.398	0.454	0.249	0.309	0.284	0.362	0.249	0.309
	336	**0.292**	**0.333**	0.311	0.348	0.305	0.343	0.597	0.542	0.325	0.366	0.339	0.372	0.312	0.346	0.568	0.555	0.309	0.347	0.369	0.427	0.321	0.351
	720	**0.390**	**0.392**	0.412	0.407	0.402	0.400	1.730	1.042	0.421	0.415	0.433	0.432	0.414	0.403	1.072	0.767	0.416	0.408	0.554	0.522	0.408	0.403
	Avg	**0.272**	**0.318**	0.288	0.332	0.281	0.326	0.757	0.610	0.305	0.349	0.327	0.371	0.288	0.330	0.570	0.530	0.289	0.333	0.350	0.401	0.291	0.333
ETTh1	96	**0.373**	**0.394**	0.386	0.405	0.414	0.419	0.423	0.448	0.376	0.419	0.449	0.459	0.390	0.411	0.440	0.450	0.401	0.412	0.386	0.400	0.384	0.402
	192	0.426	**0.422**	0.441	0.436	0.460	0.445	0.471	0.474	**0.420**	0.448	0.500	0.482	0.442	0.442	0.449	0.433	0.452	0.442	0.437	0.432	0.436	0.429
	336	0.461	**0.438**	0.487	0.458	0.501	0.466	0.570	0.546	**0.459**	0.465	0.521	0.496	0.480	0.468	0.598	0.554	0.492	0.463	0.481	0.459	0.491	0.469
	720	**0.464**	**0.458**	0.503	0.491	0.500	0.488	0.653	0.621	0.506	0.507	0.514	0.512	0.494	0.488	0.685	0.620	0.507	0.490	0.519	0.516	0.521	0.500
	Avg	**0.431**	**0.428**	0.454	0.447	0.469	0.454	0.529	0.522	0.440	0.460	0.496	0.487	0.452	0.452	0.543	0.514	0.463	0.452	0.456	0.452	0.458	0.450
ETTh2	96	**0.291**	**0.343**	0.297	0.349	0.302	0.348	0.745	0.584	0.358	0.397	0.346	0.388	0.328	0.371	0.496	0.509	0.319	0.361	0.333	0.387	0.340	0.374
	192	**0.369**	**0.393**	0.380	0.400	0.388	0.400	0.877	0.656	0.429	0.439	0.456	0.452	0.402	0.414	0.716	0.616	0.402	0.410	0.477	0.476	0.402	0.414
	336	**0.387**	**0.412**	0.428	0.432	0.426	0.433	1.043	0.731	0.496	0.487	0.482	0.486	0.435	0.443	0.718	0.614	0.444	0.446	0.594	0.541	0.452	0.452
	720	0.419	**0.438**	0.427	0.445	0.431	0.446	1.104	0.763	0.463	0.474	0.515	0.511	**0.417**	0.441	1.161	0.791	0.441	0.450	0.831	0.657	0.462	0.468
	Avg	**0.367**	**0.397**	0.383	0.407	0.387	0.407	0.942	0.684	0.437	0.449	0.450	0.459	0.396	0.417	0.773	0.633	0.402	0.417	0.559	0.515	0.414	0.427
Electricity	96	**0.147**	**0.237**	0.148	0.240	0.181	0.270	0.219	0.314	0.193	0.308	0.201	0.317	0.165	0.274	0.211	0.305	0.157	0.260	0.197	0.282	0.168	0.272
	192	**0.161**	**0.250**	0.162	0.253	0.188	0.274	0.231	0.322	0.201	0.315	0.222	0.334	0.184	0.292	0.225	0.319	0.173	0.274	0.196	0.285	0.184	0.289
	336	**0.178**	**0.268**	**0.178**	0.269	0.204	0.293	0.246	0.337	0.214	0.329	0.231	0.338	0.195	0.302	0.247	0.340	0.192	0.295	0.209	0.301	0.198	0.300
	720	**0.216**	**0.303**	0.225	0.317	0.246	0.324	0.280	0.363	0.246	0.355	0.254	0.361	0.231	0.332	0.287	0.373	0.223	0.318	0.245	0.333	0.220	0.320
	Avg	**0.176**	**0.265**	0.178	0.270	0.205	0.290	0.244	0.334	0.214	0.327	0.227	0.338	0.194	0.300	0.243	0.334	0.186	0.287	0.212	0.300	0.193	0.295
Traffic	96	0.410	**0.263**	**0.395**	0.268	0.462	0.295	0.522	0.290	0.587	0.366	0.613	0.388	0.610	0.349	0.574	0.312	0.493	0.336	0.650	0.396	0.593	0.321
	192	0.426	**0.272**	**0.417**	0.276	0.466	0.296	0.530	0.293	0.604	0.373	0.616	0.382	0.635	0.370	0.587	0.315	0.497	0.351	0.598	0.370	0.617	0.336
	336	0.435	**0.275**	**0.433**	0.283	0.482	0.304	0.558	0.305	0.621	0.383	0.622	0.337	0.670	0.393	0.594	0.318	0.528	0.361	0.605	0.373	0.629	0.336
	720	0.469	**0.296**	**0.467**	0.302	0.514	0.322	0.589	0.328	0.626	0.382	0.660	0.408	0.727	0.422	0.612	0.322	0.569	0.380	0.645	0.394	0.640	0.350
	Avg	0.435	**0.277**	**0.428**	0.282	0.481	0.304	0.550	0.304	0.610	0.376	0.628	0.379	0.661	0.384	0.592	0.317	0.522	0.357	0.625	0.383	0.620	0.336
ILI	24	1.807	**0.827**	**1.728**	0.848	2.066	0.854	3.041	1.186	2.687	1.147	3.101	1.238	2.441	0.978	4.265	1.387	2.930	1.155	2.398	1.040	2.317	0.934
	36	**1.610**	**0.791**	1.677	0.817	2.173	0.881	3.406	1.232	2.887	1.160	3.397	1.270	2.336	0.948	4.777	1.496	3.061	1.160	2.646	1.088	1.972	0.920
	48	2.071	0.883	**1.634**	**0.824**	2.032	0.892	3.459	1.221	2.797	1.155	2.947	1.203	2.429	0.989	5.333	1.592	3.081	1.178	2.614	1.086	2.238	0.940
	60	**1.807**	**0.851**	2.001	0.926	1.987	0.899	3.640	1.305	2.809	1.163	3.019	1.202	3.003	1.196	5.070	1.552	3.322	1.254	2.804	1.146	2.027	0.928
	Avg	1.824	**0.838**	**1.760**	0.854	2.065	0.882	3.387	1.236	2.795	1.156	3.116	1.228	2.552	1.028	4.861	1.507	3.099	1.187	2.616	1.090	2.139	0.931
Weather	96	0.164	**0.209**	0.174	0.214	0.177	0.218	**0.158**	0.230	0.217	0.296	0.266	0.336	0.163	0.212	0.171	0.231	0.166	0.210	0.196	0.255	0.172	0.220
	192	0.209	**0.250**	0.221	0.254	0.225	0.259	**0.206**	0.277	0.276	0.336	0.307	0.367	0.212	0.254	0.215	0.274	0.215	0.256	0.237	0.296	0.219	0.261
	336	**0.264**	**0.291**	0.278	0.296	0.278	0.297	0.272	0.335	0.339	0.380	0.359	0.395	0.272	0.299	0.266	0.313	0.287	0.300	0.283	0.335	0.280	0.306
	720	**0.340**	**0.342**	0.358	0.347	0.354	0.348	0.398	0.418	0.403	0.428	0.419	0.428	0.350	0.348	0.344	0.375	0.355	0.348	0.345	0.381	0.365	0.359
	Avg	**0.244**	**0.273**	0.258	0.278	0.259	0.281	0.259	0.315	0.309	0.360	0.338	0.382	0.249	0.278	0.249	0.298	0.256	0.279	0.265	0.317	0.259	0.287
Solar Energy	96	**0.199**	**0.230**	0.203	0.237	0.234	0.286	0.310	0.331	0.242	0.342	0.884	0.711	0.243	0.277	0.208	0.265	0.221	0.275	0.290	0.378	0.250	0.292
	192	**0.229**	**0.251**	0.233	0.261	0.267	0.310	0.734	0.725	0.285	0.380	0.834	0.692	0.267	0.295	0.236	0.293	0.268	0.306	0.320	0.398	0.296	0.318
	336	**0.244**	**0.264**	0.248	0.273	0.290	0.315	0.750	0.735	0.282	0.376	0.941	0.723	0.318	0.329	0.247	0.301	0.272	0.294	0.353	0.415	0.319	0.330
	720	**0.246**	**0.267**	0.249	0.275	0.289	0.317	0.769	0.765	0.357	0.427	0.882	0.717	0.321	0.332	0.253	0.307	0.281	0.313	0.356	0.413	0.338	0.337
	Avg	**0.230**	**0.253**	0.233	0.262	0.270	0.307	0.641	0.639	0.291	0.381	0.885	0.711	0.287	0.308	0.236	0.292	0.260	0.297	0.330	0.401	0.301	0.319
Flight	96	**0.143**	**0.251**	0.148	0.257	0.150	0.265	0.154	0.263	0.198	0.317	0.204	0.319	0.183	0.301	0.196	0.316	0.151	0.261	0.221	0.337	0.237	0.350
	192	**0.144**	**0.250**	0.149	0.257	0.150	0.263	0.159	0.265	0.203	0.324	0.200	0.314	0.189	0.306	0.272	0.379	0.158	0.268	0.220	0.336	0.224	0.337
	336	**0.154**	**0.259**	0.161	0.269	0.160	0.273	0.167	0.274	0.214	0.332	0.201	0.318	0.206	0.320	0.260	0.369	0.174	0.284	0.229	0.342	0.289	0.394
	720	**0.192**	**0.293**	0.201	0.305	0.194	0.306	0.720	0.637	0.269	0.383	0.345	0.426	0.253	0.358	0.390	0.449	0.239	0.337	0.263	0.366	0.310	0.408
	Avg	**0.158**	**0.263**	0.165	0.272	0.164	0.277	0.300	0.360	0.221	0.339	0.238	0.344	0.208	0.321	0.280	0.378	0.181	0.288	0.233	0.345	0.265	0.372
Tesla Stock	24	**0.114**	**0.229**	0.120	0.233	0.115	0.230	0.458	0.567	0.135	0.272	0.200	0.351	0.147	0.285	0.124	0.263	0.121	0.246	0.146	0.285	0.135	0.256
	36	**0.132**	**0.255**	0.138	0.257	0.137	0.260	0.359	0.521	0.153	0.291	0.202	0.346	0.159	0.296	0.145	0.291	0.138	0.264	0.144	0.277	0.157	0.281
	48	**0.145**	**0.273**	0.150	**0.273**	0.148	0.281	0.373	0.448	0.170	0.320	0.171	0.315	0.164	0.304	0.158	0.312	0.156	0.290	0.152	0.288	0.170	0.304
	60	**0.150**	0.286	0.156	**0.285**	0.152	0.291	0.499	0.606	0.161	0.311	0.206	0.357	0.161	0.300	0.179	0.334	0.173	0.311	0.158	0.296	0.182	0.321
	Avg	**0.135**	**0.261**	0.141	0.262	0.138	0.265	0.422	0.536	0.155	0.299	0.195	0.342	0.158	0.296	0.152	0.300	0.147	0.278	0.150	0.286	0.161	0.291
1st Count		**42**	**53**	9	3	0	1	2	0	2	0	0	0	1	0	0	0	0	0	0	0	0	0

Table 3
Ablation study on difference embedding and multi-head graph learning mechanism.

Datasets		ETTh1				Avg	ETTm2				Avg	Weather				Avg
Prediction Length T		96	192	336	720		96	192	336	720		96	192	336	720	
DiM	MSE	**0.373**	**0.426**	**0.461**	**0.464**	**0.431**	**0.172**	**0.235**	**0.292**	**0.390**	**0.272**	**0.164**	**0.209**	**0.264**	**0.340**	**0.244**
	MAE	**0.394**	**0.422**	**0.438**	**0.458**	**0.428**	**0.253**	**0.295**	**0.333**	**0.392**	**0.318**	**0.209**	**0.250**	**0.291**	**0.342**	**0.273**
w/o-DE	MSE	0.376	0.432	0.475	0.470	0.438	0.176	0.239	0.295	0.394	0.276	0.166	0.211	0.266	**0.340**	0.246
	MAE	0.397	0.426	0.446	0.466	0.434	0.256	0.299	0.336	0.396	0.322	0.210	0.251	0.292	**0.342**	0.274
w/o-MG	MSE	0.374	0.427	0.467	0.468	0.434	0.177	0.240	0.298	0.404	0.280	0.167	0.212	0.266	0.343	0.247
	MAE	**0.394**	**0.422**	0.441	0.464	0.430	0.260	0.302	0.339	0.399	0.325	0.210	**0.250**	0.292	0.344	0.274
w/o-DE & MG	MSE	0.377	0.432	0.470	0.478	0.439	0.181	0.244	0.301	0.405	0.283	0.191	0.236	0.288	0.360	0.269
	MAE	0.397	0.426	0.444	0.472	0.435	0.264	0.306	0.343	0.402	0.329	0.229	0.267	0.304	0.352	0.288

of the series representations d is chosen from $\{128, 256, 512\}$. For the encoder in the DiM model, we configure the number of encoder layers L to 2 for the Electricity and Solar Energy datasets, and 6 for the Traffic dataset, ensuring adequate capacity for these larger datasets. For the remaining datasets, we set the encoder layers to 1 to optimize efficiency. In addition, for the multi-head graph learning mechanism, we typically set the number of heads N to either 4 or 8, while the depth of message passing K is consistently maintained at 4.

4.2. Main results

Table 2 shows the MTS prediction results for the proposed DiM and other baseline models, where the best results are in bold and the second best results are underlined. Lower MSE/MAE represents more accurate prediction results. Overall, compared to other baseline models, DiM demonstrates significant predictive superiority, which can be consistently observed across all forecasting horizons and datasets. Specifically, DiM achieved the best performance in 42 out of 55 MSE evaluations and 53 out of 55 MAE evaluations, significantly surpassing the current SOTA model, iTransformer. Notably, although iTransformer demonstrates excellent predictive performance on large datasets such as Traffic and Electricity, its performance on smaller datasets like ETT struggles to surpass PatchTST. In contrast, our proposed DiM achieves significant improvements on smaller datasets while maintaining robust performance on larger datasets.

On the other hand, in comparison to previous GNN-based models like MTGNN and MSGNet, DiM consistently delivers superior performance across all datasets. For example, on the ETTh2 dataset, DiM improves the average MSE error from 0.396 to 0.367, representing a reduction of approximately 7.3 %, while on the Traffic dataset, DiM achieves a significant relative decrease of 26.5 % in average MSE, lowering the error from 0.592 to 0.435. These results highlight the effectiveness of the proposed multi-head graph learning mechanism in capturing channel dependencies, which is particularly crucial for large datasets, where robust channel relationships play a vital role.

4.3. Ablation study

In this section, to validate the effectiveness of the difference embedding and multi-head graph learning mechanism, we conduct ablation studies on the ETTh1, ETTm2, and Weather datasets. The experimental results are presented in Table 3, where for simplicity, the multi-head graph learning mechanism is represented by 'MG' and the difference embedding is denoted by 'DE'. Our findings reveal that the DiM, which utilizes difference embedding and multi-head graph learning mechanism, consistently outperforms others in terms of MSE and MAE. This highlights the importance of these modules in improving forecast accuracy. Notably, excluding difference embedding significantly increases error rates, particularly for longer forecast periods, emphasizing its role in capturing temporal patterns. Likewise, omitting the multi-head graph learning mechanism also results in a noticeable reduction in performance, indicating its effectiveness in capturing complex inter-channel relationships. The greatest reduction in model accuracy occurs when both modules are removed, clearly illustrating their synergistic influence on the model's success. These results affirm the critical importance of integrating both difference embedding and multi-head graph learning to enhance long-term forecasting precision.

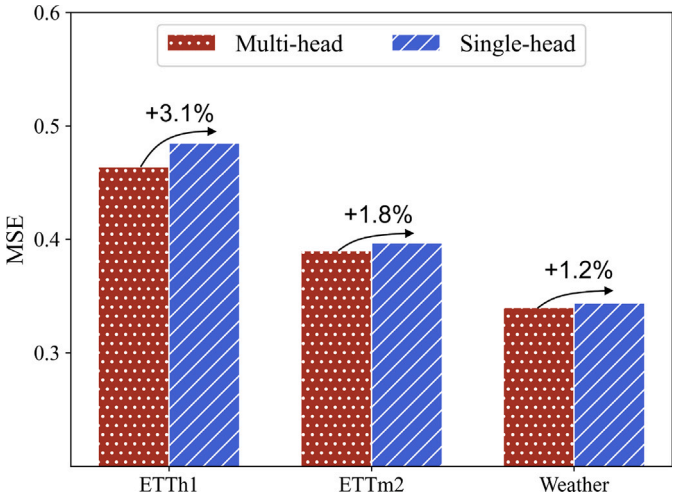


Fig. 4. Performance comparison between multi-head graph learning mechanism and traditional graph learning.

4.4. Advantages of multi-head graph learning mechanism

4.4.1. Comparison with traditional graph learning

By mapping high-dimensional representations into smaller subspaces, the multi-head graph learning mechanism enables efficient message passing within each graph, thereby reducing the model's parameter size while maintaining robust learning. To validate this, we conduct experiments from two perspectives: predictive performance and model scale. Fig. 4 displays the MSE scores for the multi-head graph learning mechanism compared to single-head graph learning, i.e., $N = 1$, across the ETTh1, ETTm2, and Weather datasets at a prediction horizon of 720 steps. It can be observed that the multi-head graph learning mechanism provides a steady performance improvement, with a 3.1 % reduction in MSE on the ETTh1 dataset, a 1.8 % reduction on ETTm2, and a 1.2 % reduction on the Weather dataset. These reductions in MSE highlight the mechanism's effectiveness in capturing complex inter-channel relationships more accurately than traditional graph learning. Additionally, the multi-head graph learning mechanism significantly reduces computational overhead, as shown in Fig. 5. As the embedding dimensions augment, traditional graph learning requires a substantial increase in model parameters, whereas the multi-head graph learning mechanism results in a reduction of approximately 50 % in parameters compared to the single-head approach. This efficiency gain demonstrates that the

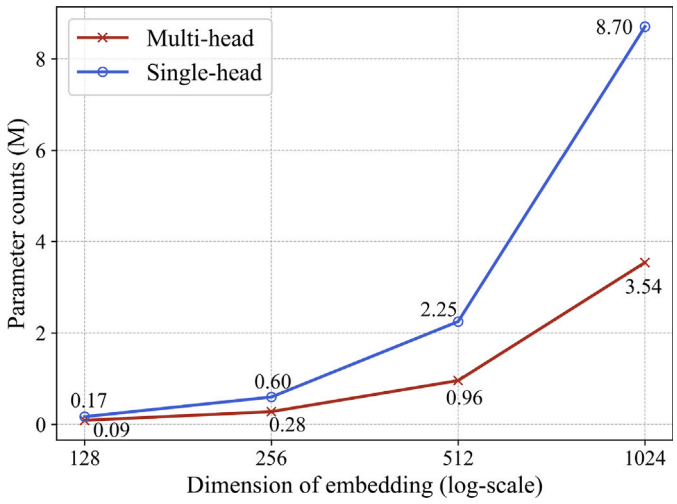


Fig. 5. Comparison of parameter counts between multi-head graph learning mechanism and traditional graph learning under varying embedding dimensions. Parameter count is measured in millions.

multi-head strategy can achieve better scalability without compromising predictive accuracy.

4.4.2. Comparison with other inter-channel modeling methods

To confirm the superiority of the multi-head graph learning mechanism, we replace it with MLP, CNN, attention modules, respectively. Additionally, we include comparisons with the channel-independent strategy (CI), an emerging paradigm that has gained prominence in MTS forecasting. For a fair comparison, we exclude the influence of the difference embedding, focusing solely on each module's ability to capture channel dependencies. The results, as depicted in Table 4, indicate that the multi-head graph learning mechanism generally achieves the best performance, confirming its robustness in unraveling complex inter-channel relationships. Interestingly, the attention mechanism exhibits inferior performance compared to the straightforward MLP and CNN. This phenomenon can be attributed to the challenges of training dynamic weights effectively on smaller datasets, a problem similarly noted in the field of computer vision [8]. However, it is important to note that the attention mechanism demonstrates a significant advantage over MLP on larger datasets, a finding that has been validated in [21]. In addition, it is noteworthy that the channel-independent strategy demonstrates superior performance over MLP, CNN, and attention mechanism on the ETTh1 dataset. This advantage stems from the inherently weak inter-channel correlations exhibited by ETTh1, where conventional relational modeling approaches tend to amplify spurious noise in cross-channel interactions rather than extract meaningful dependencies. Conversely, our multi-head graph attention mechanism maintains robust performance under such conditions, benefiting from its adaptive capability to capture both strong and weak inter-channel dependencies.

Table 5
Computational efficiency comparison between multi-head graph learning mechanism and attention mechanism on ETTm2 dataset. The forecasting horizon is set to 96. Peak memory is measured in Megabytes (MB), and parameter counts are in millions (M).

Models	Variant	Params	GFLOPs	Peak Mem
DiM	Full	2.524	0.579	62.2
	w/o MG	2.251	0.505	55.3
	MG	0.273	0.074	6.9
iTransformer	Full	3.252	0.730	74.1
	w/o Attention	2.201	0.493	53.7
	Attention	1.051	0.237	20.4

4.4.3. Computational efficiency analysis of multi-head graph learning mechanism

To quantitatively assess the computational overhead of the proposed multi-head graph learning mechanism and evaluate its performance-cost trade-off, we conduct rigorous efficiency comparisons on the ETTm2 dataset. Table 5 documents detailed measurements of model size (parameters), computational requirements (GFLOPs), and memory consumption (peak memory) across three configurations: (1) full DiM and iTransformer frameworks, (2) their ablated variants without respective relation modeling components, and (3) standalone MG and attention modules. As presented in Table 5, the MG component itself requires only 0.074 GFLOPs, incurring approximately 12 % computational overhead within the DiM framework. This is significantly lower than the 32 % overhead attributed to the attention mechanism in iTransformer, indicating a practically manageable computational burden. Moreover, compared to conventional attention mechanisms, MG reduces parameter requirements and computational costs by approximately 70 %, enabling substantially more lightweight modeling of inter-channel relationships while achieving better modeling effectiveness.

4.5. Advantages of DI embedding

4.5.1. Comparison with other embedding methods

To rigorously evaluate the contribution of our proposed DI embedding, we conduct comprehensive ablation studies comparing its forecasting performance and computational efficiency against

Table 4
Comparison with other inter-channel modeling methods and the channel-independent strategy.

Datasets		ETTh1				Avg	ETTm2				Avg	Weather				Avg
Prediction Length T		96	192	336	720		96	192	336	720		96	192	336	720	
MG	MSE	**0.376**	**0.432**	0.475	**0.470**	**0.438**	**0.176**	**0.239**	**0.295**	**0.394**	**0.276**	**0.164**	**0.209**	**0.264**	**0.340**	**0.244**
	MAE	**0.397**	**0.426**	0.446	**0.466**	**0.434**	**0.256**	**0.299**	**0.336**	**0.396**	**0.322**	**0.209**	**0.250**	**0.291**	**0.342**	**0.273**
MLP	MSE	0.377	0.434	0.473	0.489	0.443	0.178	0.240	0.304	0.402	0.281	0.172	0.219	0.275	0.349	0.254
	MAE	0.398	0.429	0.447	0.481	0.439	0.262	0.304	0.345	0.403	0.329	0.218	0.257	0.298	0.348	0.280
CNN	MSE	0.384	0.441	0.480	0.493	0.450	0.181	0.245	0.308	0.412	0.287	0.183	0.231	0.283	0.355	0.263
	MAE	0.405	0.435	0.453	0.483	0.444	0.265	0.308	0.345	0.406	0.331	0.223	0.262	0.300	0.348	0.283
Attention	MSE	0.380	0.434	**0.470**	0.492	0.444	0.183	0.250	0.320	0.412	0.291	0.179	0.228	0.282	0.356	0.261
	MAE	0.401	0.431	0.446	0.484	0.441	0.265	0.311	0.353	0.408	0.334	0.218	0.261	0.300	0.350	0.282
CI	MSE	0.377	**0.432**	**0.470**	0.478	0.439	0.181	0.244	0.301	0.405	0.283	0.191	0.236	0.288	0.360	0.269
	MAE	**0.397**	**0.426**	**0.444**	0.472	0.435	0.264	0.306	0.343	0.402	0.329	0.229	0.267	0.304	0.352	0.288

Table 6
Comprehensive comparison of embedding methods including accuracy and efficiency metrics. Parameter counts are in millions (M).

Methods		Vanilla				Dimension-invariant				DI (Ours)			
Metric		MSE	MAE	GFLOPs	Params	MSE	MAE	GFLOPs	Params	MSE	MAE	GFLOPs	Params
ETTh1	96	0.865	0.713	5.305	0.339	0.391	0.397	0.966	0.207	**0.382**	**0.397**	**0.241**	**0.170**
	192	1.008	0.792	7.447	0.339	0.442	0.430	1.036	0.256	**0.435**	**0.427**	**0.258**	**0.182**
	336	1.107	0.809	10.644	0.339	0.487	0.449	1.139	0.330	**0.476**	**0.446**	**0.284**	**0.201**
	720	1.181	0.865	19.209	0.339	0.513	0.486	1.416	0.527	**0.499**	**0.480**	**0.353**	**0.251**
	Avg	1.040	0.795	10.651	0.339	0.458	0.441	1.139	0.355	**0.448**	**0.438**	**0.284**	**0.201**
ETTm2	96	0.365	0.453	28.458	7.388	**0.177**	**0.261**	4.655	3.449	0.179	0.264	**1.164**	**3.302**
	192	0.533	0.563	39.860	7.388	**0.243**	0.305	4.724	3.646	**0.243**	**0.303**	**1.181**	**3.351**
	336	1.363	0.887	56.949	7.388	0.311	0.348	4.828	3.941	**0.306**	**0.346**	**1.207**	**3.425**
	720	3.379	1.338	102.559	7.388	0.408	0.405	5.105	4.728	**0.406**	**0.402**	**1.276**	**3.622**
	Avg	1.410	0.810	56.902	7.388	0.285	0.330	4.828	3.941	**0.284**	**0.329**	**1.207**	**3.425**
Weather	96	0.300	0.384	17.067	1.096	0.171	0.216	6.073	0.544	**0.167**	**0.211**	**1.516**	**0.471**
	192	0.598	0.544	23.922	1.096	0.216	0.256	6.387	0.643	**0.215**	**0.253**	**1.595**	**0.495**
	336	0.578	0.523	34.204	1.096	0.273	0.296	6.589	0.790	**0.271**	**0.294**	**1.713**	**0.532**
	720	1.059	0.741	61.704	1.096	**0.349**	0.347	8.118	1.184	0.350	**0.345**	**2.028**	**0.631**
	Avg	0.634	0.548	34.224	1.096	0.252	0.279	6.589	0.790	**0.251**	**0.276**	**1.713**	**0.532**

Table 7
Performance improvement achieved by applying the proposed DI embedding to Transformer variant models. The selected baseline models, iInformer, iFlowformer, and iFlashformer, are reported in [21].

Models		iInformer		+ **DE**		iFlowformer		+ **DE**		iFlashformer		+ **DE**	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.387	0.402	**0.382**	**0.397**	0.383	0.400	**0.382**	**0.397**	0.390	0.403	**0.384**	**0.398**
	192	0.442	0.433	**0.435**	**0.427**	0.445	0.434	**0.438**	**0.429**	**0.437**	0.430	**0.437**	**0.429**
	336	0.488	0.456	**0.476**	**0.446**	0.495	0.426	**0.483**	**0.452**	0.486	0.456	**0.476**	**0.448**
	720	0.521	0.496	**0.499**	**0.480**	0.514	0.493	**0.499**	**0.483**	0.506	0.490	**0.498**	**0.485**
	Avg	0.460	0.447	**0.448**	**0.438**	0.459	0.438	**0.451**	**0.440**	0.455	0.445	**0.449**	**0.440**
ETTm2	96	0.180	0.265	**0.179**	**0.264**	0.183	0.269	**0.180**	**0.266**	0.182	0.266	**0.180**	**0.264**
	192	0.245	0.305	**0.243**	**0.303**	0.248	0.309	**0.245**	**0.306**	0.250	0.310	**0.246**	**0.307**
	336	0.309	0.347	**0.306**	**0.346**	0.314	0.352	**0.312**	**0.349**	0.322	0.356	**0.315**	**0.351**
	720	0.410	0.404	**0.406**	**0.402**	0.424	0.416	**0.409**	**0.404**	0.420	0.412	**0.411**	**0.404**
	Avg	0.286	0.330	**0.284**	**0.329**	0.292	0.337	**0.287**	**0.331**	0.294	0.336	**0.288**	**0.332**
Weather	96	0.174	0.217	**0.167**	**0.211**	0.186	0.224	**0.178**	**0.219**	0.182	0.222	**0.172**	**0.216**
	192	0.220	0.258	**0.215**	**0.253**	0.233	0.263	**0.218**	**0.256**	0.231	0.263	**0.216**	**0.254**
	336	0.275	0.297	**0.271**	**0.294**	0.314	0.352	**0.312**	**0.349**	0.286	0.302	**0.273**	**0.295**
	720	0.352	0.347	**0.350**	**0.345**	0.424	0.416	**0.409**	**0.404**	0.360	0.351	**0.349**	**0.345**
	Avg	0.255	0.280	**0.251**	**0.276**	0.289	0.314	**0.279**	**0.307**	0.265	0.285	**0.253**	**0.278**

two prominent alternatives: vanilla embedding [45] and dimension-invariant embedding [41]. Using Informer as the base model architecture, we measure prediction accuracy and resource requirements across three benchmark datasets under identical experimental settings. Results in Table 6 demonstrate that the proposed DI embedding achieves dramatic performance improvements over vanilla embedding, reducing average MSE by approximately 65 % while requiring comparable computational resources. Moreover, compared to dimension-invariant embedding, DI embedding achieves better forecasting performance with substantially superior efficiency, requiring approximately 4× fewer computational operations (GFLOPs) and a meaningful reduction in parameter requirements. These results demonstrate that the proposed DI embedding method strikes an optimal balance between forecasting accuracy and computational efficiency, thereby establishing a practical representation paradigm for MTS.

4.5.2. DI embedding generality

Our proposed DI embedding method is able to serve as a versatile, plug-and-play module to enhance the temporal information extraction capability of models. To verify this, we replace the inverted embedding used in iInformer, iFlowformer, and iFlashformer with DI embedding. These models, which originally replace the vanilla embedding with inverted embedding, have been reported in literature [21]. As shown in Table 7, DI embedding consistently improves the predictive performance of various models, demonstrating its effectiveness and broad applicability in capturing and leveraging temporal patterns. More importantly, the computational overhead introduced by the differencing operation is negligible, allowing DI embedding to improve model performance without significantly increasing the computational burden, making it highly efficient and practical.

4.6. Effect of hyperparameters

In this section, we investigate the effects of four key hyperparameters on the DiM: (1) the proportion of DI embedding, (2) the depth of graph propagation, (3) the embedding dimension, and (4) the number of encoder layers. We conduct experiments on the ETTm2 dataset at four distinct prediction horizons, and the results are presented in Fig. 6. Typically, DiM consistently exhibits stable performance, irrespective of hyperparameter settings.

Ratio of DI embedding [Fig. 6(a)]: When $\rho = 0.25, 0.5$, or 0.75, the model's performance remains consistently stable, suggesting that DiM is not overly sensitive to this particular hyperparameter. To ensure a balanced representation between difference embedding and inverted

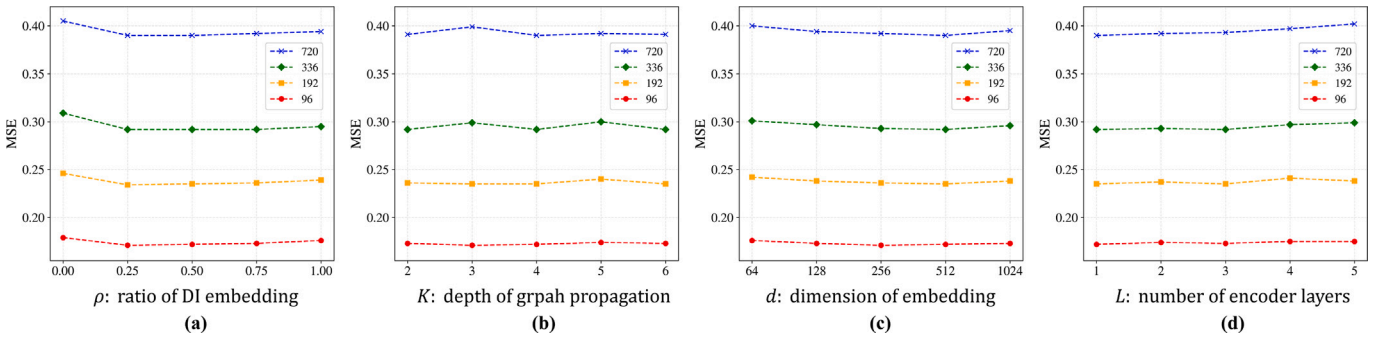


Fig. 6. Hyperparameter sensitivity.

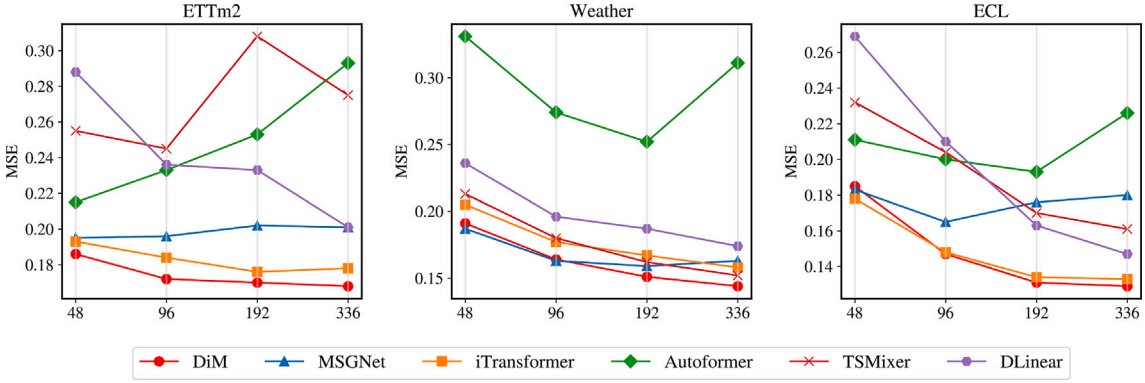


Fig. 7. Forecasting performance with varying lookback lengths $S \in \{48, 96, 192, 336\}$ and the fixed prediction length $T = 96$.

Table 8
Computational efficiency comparison on ETTm2 dataset. Peak memory is measured in Megabytes (MB), and parameter counts are in millions (M).

Models	$T = 96$			$T = 720$		
	GFLOPs	Peak Mem	Params	GFLOPs	Peak Mem	Params
Autoformer	28.370	848.2	7.388	102.246	3415.6	7.388
FEDformer	24.075	1240.3	13.670	87.482	2504.2	13.670
Crossformer	40.461	961.1	25.311	217.306	3036.4	25.551
PatchTST	8.664	236.8	3.752	9.522	299.2	7.586
iTransformer	0.730	74.1	3.252	0.802	84.1	3.572
MTGNN	12.199	463.2	4.583	12.217	467.4	4.663
MSGNet	51.720	1090.2	4.797	51.734	1094.1	4.918
DiM (Ours)	**0.579**	**62.2**	**2.524**	**0.650**	**69.1**	**2.845**

embedding, and to simplify our model configuration for broader applicability, we choose $\rho = 0.5$ as the standard setting throughout this paper.

Depth of graph propagation [Fig. 6(b)]: Deeper graph propagation indicates that each node can aggregate information from farther distances, but it may also introduce noise and increase computational complexity. We observe that the model performs best when $K = 4$, therefore, we set the depth of graph propagation to 4.

Dimension of embedding [Fig. 6(c)]: The embedding dimension directly influences the feature representation capacity of the model. Generally, larger embedding dimensions offer better representation capacity but may lead to a higher risk of overfitting. Here, we set the embedding dimension to 512, as it demonstrates better performance across all four prediction horizons.

Number of encoder layers [Fig. 6(d)]: Increasing the number of encoder layers raises computational costs and memory usage. Since additional layers do not yield a significant decrease in MSE, we opt to set the number of layers to $L = 1$.

4.7. Increasing lookback window length

In theory, a longer lookback window increases the receptive field, which generally improves the forecasting performance. However, excessively long inputs may lead to overfitting, preventing the model from efficiently extracting temporal information. To explore the impact of an extended lookback window on forecasting performance, we vary the input length from 48 to 336 on the ETTm2, Electricity, and weather datasets. As shown in Fig. 7, the MSE scores of the DiM steadily decrease as the lookback window enlarges, indicating that DiM is capable of capturing more valuable temporal information within an extended lookback window. Additionally, in the context of longer lookback windows, DiM consistently outperforms other models, further validating its robustness in handling longer sequences. Moreover, it can be observed that the predictive performance of Autoformer and MSGNet deteriorates with an extended lookback window. This decline can be attributed to the limitations of the vanilla embedding employed by these models, which fails to effectively leverage longer sequences.

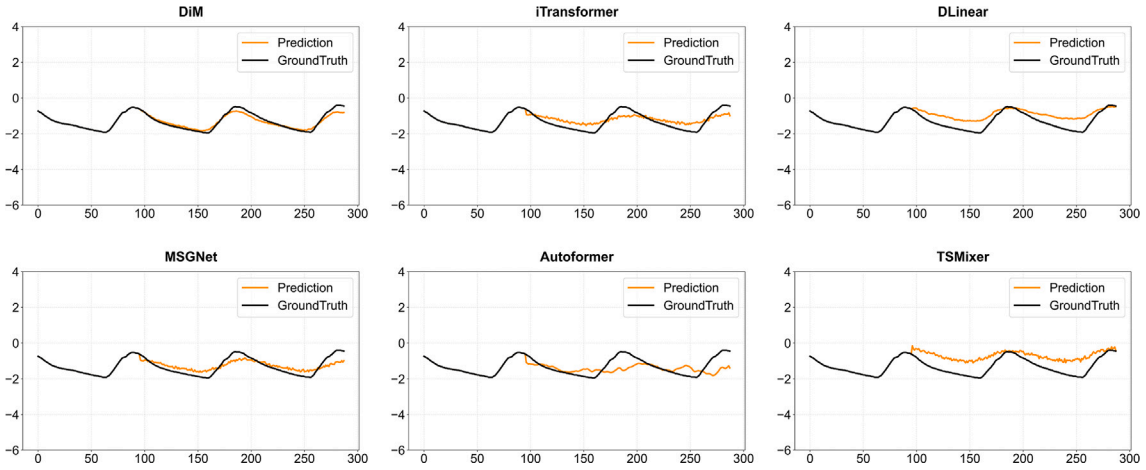


Fig. 8. Visualization of forecasting results on the ETTm2 dataset with a fixed prediction length of 192.

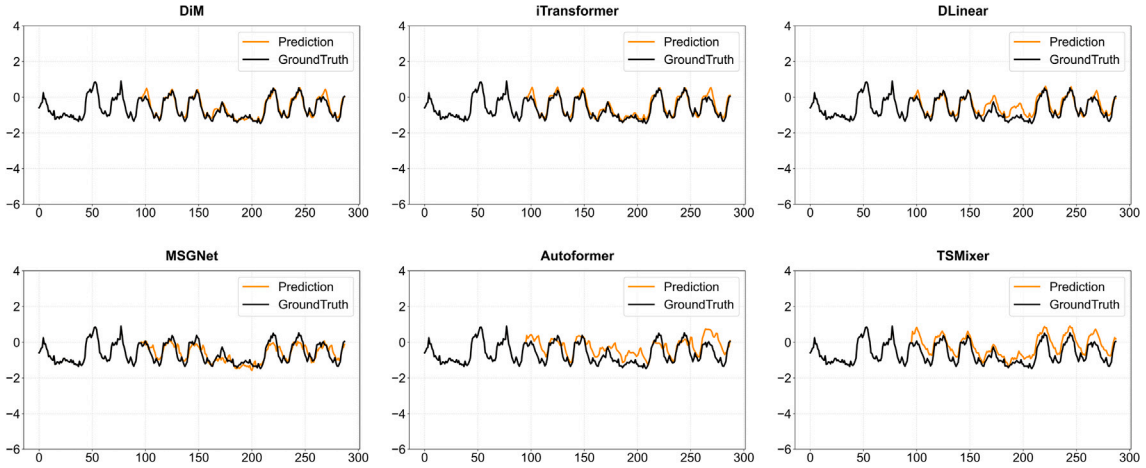


Fig. 9. Visualization of forecasting results on the Electricity dataset with a fixed prediction length of 192.

4.8. Model efficiency

To further evaluate the computational efficiency of the DiM, we conduct a comparative analysis against several state-of-the-art baselines. These include Transformer-based models (Autoformer, FEDformer, Crossformer, PatchTST, and iTransformer) and GNN-based models (MTGNN and MSGNet), which are considered the most relevant to DiM. As reported in Table 8, DiM consistently demonstrates superior efficiency across both short-term ($T = 96$) and long-term ($T = 720$) forecasting horizons. Among the Transformer-based baselines, Autoformer and FEDformer employ vanilla embedding strategies, where all variables at each time step are treated as a single token. While this design is straightforward, it incurs significant computational overhead, particularly when handling long input sequences. In contrast, Crossformer and PatchTST adopt patch embedding techniques to reduce the effective sequence length, thereby improving efficiency to some extent. Nevertheless, their computational demands are still significant, limiting their scalability in practice. DiM outperforms all these models by a substantial margin. For instance, when $T = 720$, DiM requires only 0.650 GFLOPs and 69.1 MB of peak memory, in comparison to 102.2 GFLOPs / 3415.6 MB for Autoformer and 217.3 GFLOPs / 3036.4 MB for Crossformer. Compared to iTransformer, which introduces inverted embedding for efficiency, DiM still achieves lower GFLOPs and memory usage under both horizons, which can be evidenced by a 19 % reduction in GFLOPs and an 18 % reduction in memory usage at $T = 720$. This reflects the advantage of the proposed multi-head graph learning mechanism, which efficiently captures inter-channel dependencies with significantly reduced computational cost relative to attention-based counterparts. Furthermore, DiM reduces computational overhead by an order of magnitude compared to GNN-based baselines such as MTGNN and MSGNet. This significant efficiency improvement derives from two key factors: the enhanced computational efficiency of our multi-head graph learning mechanism relative to conventional graph learning methods, and the fundamental rationality of DiM's architectural design, which optimally balances forecasting performance with computational economy.

4.9. Visualization

In order to provide an intuitive comparison, we visualize the prediction results of six models on the ETTm2 and Electricity datasets, as depicted in Figs. 8 and 9. Among these models, DiM delivers predictions that are closer to the ground truth. This superiority stems partly from DiM's ability to effectively extract temporal information, allowing it to understand time series features such as cyclicity and trends. Additionally, its capacity to model inter-channel relationships enables information from other channels to effectively guide predictions for the current channel. Fig. 9 reveals that while DLinear effectively tracks regular patterns, it fails to match the ground truth during periods of volatility. This discrepancy can be attributed to its oversight of

inter-channel information, resulting in forecasts that predominantly replicate the cyclical nature of the data. Consequently, this approach struggles to identify subtle trends and abrupt changes. Besides, although MSGNet and TSMixer acknowledge the importance of inter-channel information, their insufficient robustness in modeling inter-channel relationships hinders their ability to deliver reliable forecasts.

5. Conclusion

In this paper, we propose DiM, a novel MTS forecasting framework that effectively captures inherent temporal dependencies and inter-channel relationships within MTS data. Specifically, we introduce the DI embedding to refine the model's capacity to extract temporal patterns within time series. Moreover, recognizing the dilemma that a single graph structure struggles to represent the intricate dynamic relationships between channels, we incorporate a multi-head graph learning mechanism. Our model demonstrates impressive advantages in extensive experiments and achieves SOTA performance on eleven public datasets. Additionally, DiM breaks away from the conventional Transformer paradigm by adopting the Metaformer architecture, providing a new perspective for subsequent research in MTS forecasting.

For future research, several promising directions could build upon the findings of this paper. Firstly, our results indicate that the Metaformer framework shows significant potential in MTS forecasting. This suggests a promising direction for future research to explore better token mixers within the Metaformer framework for more effective modeling of inter-channel relationships. Additionally, identifying and developing more robust embedding methods to effectively represent time series remains a significant and ongoing challenge.

CRediT authorship contribution statement

Yipeng Mo: Writing – original draft, Visualization, Software, Methodology, Conceptualization. **Haoxin Wang:** Supervision, Software, Methodology, Conceptualization. **Zuhua Yao:** Validation, Supervision, Investigation, Data curation. **Chengteng Yang:** Validation, Supervision, Investigation. **Bixiong Li:** Validation, Investigation. **Yiping Jiang:** Validation, Investigation. **Songhai Fan:** Software, Investigation. **Site Mo:** Validation, Supervision, Investigation, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Science and Technology Project of State Grid Corporation of China (Research on New Low-Curie-Temperature Magnetic Materials Technology for Transmission Line Anti-Icing, No. 521999250019–116-ZN).

Data availability

Data will be made available on request.

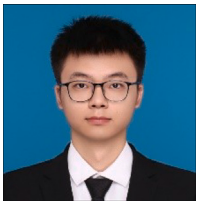
References

[1] S. Abu-El-Haija, et al., Mixhop: higher-order graph convolutional architectures via sparsified neighborhood mixing, in: PROC. INT. CONF. MACH. LEARN, 2019, pp. 21–29.
[2] S. Bai, J.Z. Kolter, V. Koltun, An empirical evaluation of generic convolutional and recurrent networks for sequence modeling, arXiv preprint arXiv:1803.01271 (2018).
[3] Box, G.E., Jenkins, G.M., Reinsel, G.C., Ljung, G.M., 2015. Time Series Analysis: Forecasting and Control Wiley, Hoboken, NJ, USA.
[4] W. Cai, Y. Liang, X. Liu, J. Feng, Y. Wu, Msgnet: learning multi-scale inter-series correlations for multivariate time series forecasting, in: Proc. AAAI Conf. Artif. Intell, 2024, pp. 11141–11149.
[5] S.A. Chen, C.L. Li, N. Yoder, S.O. Arik, T. Pfister, Tsmixer: an all-MLP architecture for time series forecasting, arXiv preprint arXiv:2303.06053 (2023).
[6] A.W.Z. Chew, Y. Pan, Y. Wang, L. Zhang, Hybrid deep learning of social media big data for predicting the evolution of COVID-19 transmission, Knowl.-Based Syst. 233 (2021) 107417.
[7] J.Y. Franceschi, A. Dieuleveut, M. Jaggi, Unsupervised scalable representation learning for multivariate time series, in: PROC. ADV. NEURAL INF. PROCESS. SYST, 2019, pp. 4652–4663.
[8] H. Gani, M. Naseer, M. Yaqub, How to train vision transformer on small-scale datasets?, arXiv preprint arXiv:2210.07240 (2022).
[9] J.D. Hamilton, Time Series Analysis, Princeton university press, 2020.
[10] L. Han, X.Y. Chen, H.J. Ye, D.C. Zhan, SOFTS: efficient multivariate time series forecasting with series-core fusion, arXiv preprint arXiv:2404.14197 (2024a).
[11] L. Han, H.J. Ye, D.C. Zhan, The capacity and robustness trade-off: revisiting the channel independent strategy for multivariate time series forecasting, IEEE Trans. Knowl. Data Eng. 36 (2024b) 7129–7142.
[12] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (1997) 1735–1780.
[13] J.D.M.W.C. Kenton, L.K. Toutanova, Bert: pre-training of deep bidirectional transformers for language understanding, in: Proc. NAACL-HLT, 2019, pp. 1–16.
[14] T. Kim, J. Kim, Y. Tae, C. Park, J.H. Choi, J. Choo, Reversible instance normalization for accurate time-series forecasting against distribution shift, in: PROC. INT. CONF. LEARN. Represent, 2022, pp. 1–25.
[15] D.P. Kingma, Adam: a method for stochastic optimization, arXiv preprint arXiv:1412.6980 (2014).
[16] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: PROC. INT. CONF. LEARN. Represent, 2017, pp. 1–14.
[17] G. Lai, W.C. Chang, Y. Yang, H. Liu, Modeling long- and short-term temporal patterns with deep neural networks, in: Proc. 41st Int. ACM SIGIR Conf. Res. Develop. Inf. Retr, 2018, pp. 95–104.
[18] J. Lei Ba, J.R. Kiros, G.E. Hinton, Layer normalization, arXiv preprint arXiv:1607.06450 (2016).
[19] Z. Li, Z. Rao, L. Pan, Z. Xu, MTS-Mixers: multivariate time series forecasting via factorized temporal and channel mixing, arXiv preprint arXiv:2302.04501 (2023).
[20] S. Liu, et al., Pyraformer: low-complexity pyramidal attention for long-range time series modeling and forecasting, in: PROC. INT. CONF. LEARN. Represent, 2022, pp. 1–20.
[21] Y. Liu, et al., iTransformer: inverted transformers are effective for time series forecasting, arXiv preprint arXiv:2310.06625 (2023).
[22] S. Mo, H. Wang, B. Li, Z. Xue, S. Fan, X. Liu, Powerformer: a temporal-based transformer model for wind power forecasting, Energy Rep. 11 (2024a) 736–744.
[23] Y. Mo, H. Wang, C. Yang, Z. Yao, B. Li, S. Fan, S. Mo, Fdnet: frequency filter enhanced dual LSTM network for wind power forecasting, Energy 312 (2024b) 133514.
[24] Y. Nie, N.H. Nguyen, P. Sinthong, J. Kalagnanam, A time series is worth 64 words: long-term forecasting with transformers, in: PROC. INT. CONF. LEARN. Represent, 2023, pp. 1–24.
[25] P. Niu, T. Zhou, X. Wang, L. Sun, R. Jin, Attention as robust representation for time series forecasting, arXiv preprint arXiv:2402.05370 (2024).
[26] A. Patton, Copula methods for forecasting multivariate time series, Handb. Econ. Forecast. 2 (2013) 899–960.
[27] F. Petropoulos, et al., Forecasting: theory and practice, Int. J. Forecast. 38 (2022) 705–871.
[28] Z. Qiu, Z. Xie, Z. Ji, X. Liu, G. Wang, Integrating query data for enhanced traffic forecasting: a spatio-temporal graph attention convolution network approach with delay modeling, Knowl.-Based Syst. 301 (2024) 112315.
[29] J. Tian, L. Han, M. Chen, Y. Xu, Z. Chen, T. Zhu, L. Sun, W. Lv, Mfgcn: multi-faceted spatial and temporal specific graph convolutional network for traffic-flow forecasting, Knowl.-Based Syst. 306 (2024) 112671.
[30] A. Vaswani, et al., Attention is all you need, in: PROC. ADV. NEURAL INF. PROCESS. SYST, 2017, pp. 1–11.
[31] H. Wang, et al., Dance of channel and sequence: an efficient attention-based approach for multivariate time series forecasting, arXiv preprint arXiv:2312.06220 (2023).
[32] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, M. Long, TimesNet: temporal 2D-variation modeling for general time series analysis, in: PROC. INT. CONF. LEARN. Represent, 2023, pp. 1–23.
[33] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: decomposition transformers with auto-correlation for long-term series forecasting, in: PROC. ADV. NEURAL INF. PROCESS. SYST, 2021, pp. 22419–22430.
[34] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, C. Zhang, Connecting the dots: multivariate time series forecasting with graph neural networks, in: Proc. 26th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min, 2020, pp. 753–763.
[35] Q. Xiong, K. Tang, M. Ma, J. Xu, T. Li, TDT loss takes it all: integrating temporal dependencies among targets into non-autoregressive time series forecasting, arXiv preprint arXiv:2406.04777 (2024).
[36] J. Yin, et al., sTransformer: a modular approach for extracting inter-sequential and temporal information for time-series forecasting, arXiv preprint arXiv:2408.09723 (2024).
[37] W. Yu, et al., Metaformer is actually what you need for vision, in: Proc. IEEE Conf. Comput. Vis. Pattern Recog, 2022, pp. 10819–10829.
[38] Y. Yu, W. Yu, F. Nie, X. Li, Prformer: pyramidal recurrent transformer for multivariate time series forecasting, arXiv preprint arXiv:2408.10483 (2024).
[39] A. Zeng, M. Chen, L. Zhang, Q. Xu, Are transformers effective for time series forecasting?, in: Proc. AAAI Conf. Artif. Intell, 2023, pp. 11121–11128.
[40] Q. Zhang, C. Li, F. Su, Y. Li, Spatiotemporal residual graph attention network for traffic flow forecasting, IEEE Internet Things J. 10 (2023) 11518–11532.

[41] Y. Zhang, R. Wu, S.M. Dascalu, F.C. Harris, Multi-scale transformer pyramid networks for multivariate time series forecasting, IEEE Access 12 (2024a) 14731–14741.
[42] Y. Zhang, J. Yan, Crossformer: transformer utilizing cross-dimension dependency for multivariate time series forecasting, in: PROC. INT. CONF. LEARN. Represent, 2023, pp. 1–21.
[43] Z. Zhang, L. Meng, Y. Gu, Sageformer: series-aware framework for long-term multivariate time-series forecasting, IEEE Internet Things J. 11 (2024b) 18435–18448.
[44] F. Zhou, Q. Yang, K. Zhang, G. Trajcevski, T. Zhong, A. Khokhar, Reinforced spatiotemporal attentive graph neural networks for traffic forecasting, IEEE Internet Things J. 7 (2020) 6414–6428.
[45] H. Zhou, et al., Informer: beyond efficient transformer for long sequence time-series forecasting, in: Proc. AAAI Conf. Artif. Intell, 2021, pp. 11106–11115.
[46] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, R. Jin, Fedformer: frequency enhanced decomposed transformer for long-term series forecasting, in: PROC. INT. CONF. LEARN. Represent, 2022, pp. 27268–27286.

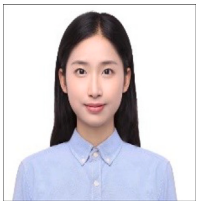
Author biography

Yipeng Mo is currently pursuing a master's degree in artificial intelligence at the College of Electrical Engineering, Sichuan University, Chengdu, China. He obtained a bachelor's degree from Zhengzhou University, Zhengzhou, China, in 2023. His research interests include time series analysis and spatial-temporal modeling, with a focus on developing algorithms for forecasting, improving IoT connectivity, and modeling complex spatial-temporal patterns in data.



Haoxin Wang is currently a master's student in control science and engineering at the College of Electrical Engineering, Sichuan University, Chengdu, China. He obtained his bachelor's degree in Automation from Xi'an University of Technology, Shannxi, China in 2022. He has been dedicated to the fields of deep learning and time series analysis, publishing several SCI papers in related areas. His work focuses on advancing spatial-temporal forecasting techniques and exploring innovative applications of artificial intelligence to real-world problems.

Zuhua Yao is currently a master's student in electronic information at the College of Electrical Engineering, Sichuan University, Chengdu, China. He received his bachelor's degree in Automation from Beijing University of Chemical Technology in 2019. And he joined the work after his bachelor's degree, working on servo motor controller research. His current research areas are deep learning and time series analysis.



Chengteng Yang is currently a master's student in electronic information at the College of Electrical Engineering, Sichuan University, Chengdu, China. She obtained her bachelor's degree in Southwest Jiaotong University, Chengdu, China, in 2023. Her research areas are deep learning and signal processing.

Bixiong Li is a full professor at the school of architecture and environment of Sichuan University, where she joined in 1995. She holds a bachelor degree in industrial and civil construction from Sichuan University at 1992, a master degree in geotechnical engineering from Sichuan University at 1995, and a PhD in Civil Engineering from Sichuan University at 2011. She was the head of the department of Civil Engineering more than 12 years. She is currently a Principal Investigator of State Key Laboratory of Intelligent Construction and Healthy Operation and Maintenance of Deep Underground Engineering at Sichuan University. Her research interests are related to artificial intelligence methods and applications on civil engineering, including data mining, deep learning, neural networks and automatic machine learning.

Jiang Yiping, born in January 1983, holds a master's degree. He currently serves as a Senior Economist, with research interests centered on high voltage and insulation technology, as well as artificial intelligence.

Songhai Fan received his PhD degree in July 2010 from Chongqing University. He is currently a senior engineer at the Electric Power Research Institute of State Grid Power Company of Sichuan Province, specializing in the condition monitoring technology of power transmission and substation equipment. He has presided over more than 10 projects at national, provincial and ministerial levels, including one project of National Natural Fund, and has been awarded the 5th batch of special grants from China Postdoctoral Science Fund and the 3rd batch of special grants from Chongqing Municipal Postdoctoral Science Fund. In recent years, he has published more than 20 academic papers, of which 7 are included in SCI core and 19 are included in EI.

Site Mo is an associate professor at the College of Electrical Engineering, Sichuan University, Chengdu, China, where he has been working since 2005. Mo Site holds a Ph.D. in electrical engineering from Sichuan University (2018). He has published more than 50 papers in specialized journals and has been granted more than 60 invention patents. Since 2010, he has participated in over 30 research projects. His experience spans the areas of Electrical Engineering and Instrument Science, with a primary focus on neural networks, signal acquisition, signal processing, digital image processing, high-speed digital cameras, IoT, and time series forecasting.